cancer-survival-gene-analysis

2026-03-16

```
library(TCGAbiolinks)
library(SummarizedExperiment)

## The following objects are masked from 'package:matrixStats':
##
##     anyMissing, rowMedians

query <- GDCquery(
  project = "TCGA-BRCA",
  data.category = "Transcriptome Profiling",
  data.type = "Gene Expression Quantification",
  workflow.type = "STAR - Counts"
)

GDCdownload(query)

## Downloading data for project TCGA-BRCA

## Of the 1231 files for download 1231 already exist.

## All samples have been already downloaded

data <- GDCprepare(query)

clinical_df_tumor <- clinical_df_tumor |>
  group_by(patient) |>
  arrange(desc(days_to_last_follow_up)) |>
  slice(1) |>
  ungroup()



sum(duplicated(clinical_df_tumor$patient))

## [1] 0

nrow(clinical_df_tumor)

## [1] 1095

clinical_df_tumor$survival_group <- ifelse(
  clinical_df_tumor$vital_status == "Dead" &
  clinical_df_tumor$follow_ups_disease_response == "WT-With Tumor",
  "Did Not Survive",
  ifelse(
    clinical_df_tumor$vital_status == "Alive" &
    clinical_df_tumor$follow_ups_disease_response == "TF-Tumor Free",
    "Survived",
    "Unknown"
```

```
  )
)

expr_matrix <- assay(data)  # raw counts



clinical_filtered <- clinical_df_tumor %>%
  filter(survival_group %in% c("Survived", "Did Not Survive"))


common_patients <- intersect(colnames(expr_matrix), clinical_filtered$barcod
e)
expr_filtered <- expr_matrix[, common_patients]
clinical_filtered <- clinical_filtered[match(common_patients, clinical_filter
ed$barcode), ]

library(DESeq2)


dds <- DESeqDataSetFromMatrix(
  countData = expr_filtered,
  colData = clinical_filtered,
  design = ~ survival_group
)

## Warning in DESeqDataSet(se, design = design, ignoreRank): some variables i
n
## design formula are characters, converting to factors

##   Note: levels of factors in the design contain characters other than
##   letters, numbers, '_' and '.'. It is recommended (but not required) to u
se
##   only letters, numbers, and delimiters '_' or '.', as these are safe char
acters
##   for column names in R. [This is a message, not a warning or an error]

keep <- rowSums(counts(dds) >= 10) >= 10
dds <- dds[keep, ]


dds$survival_group <- relevel(dds$survival_group, ref = "Did Not Survive")


dds <- DESeq(dds)

## estimating size factors
##   Note: levels of factors in the design contain characters other than
##   letters, numbers, '_' and '.'. It is recommended (but not required) to u
se
```

```
##   only letters, numbers, and delimiters '_' or '.', as these are safe char
acters
##   for column names in R. [This is a message, not a warning or an error]

## estimating dispersions

## gene-wise dispersion estimates

## mean-dispersion relationship

##   Note: levels of factors in the design contain characters other than
##   letters, numbers, '_' and '.'. It is recommended (but not required) to u
se
##   only letters, numbers, and delimiters '_' or '.', as these are safe char
acters
##   for column names in R. [This is a message, not a warning or an error]

## final dispersion estimates

## fitting model and testing

##   Note: levels of factors in the design contain characters other than
##   letters, numbers, '_' and '.'. It is recommended (but not required) to u
se
##   only letters, numbers, and delimiters '_' or '.', as these are safe char
acters
##   for column names in R. [This is a message, not a warning or an error]

## -- replacing outliers and refitting for 6975 genes
## -- DESeq argument 'minReplicatesForReplace' = 7
## -- original counts are preserved in counts(dds)

## estimating dispersions

## fitting model and testing

##   Note: levels of factors in the design contain characters other than
##   letters, numbers, '_' and '.'. It is recommended (but not required) to u
se
##   only letters, numbers, and delimiters '_' or '.', as these are safe char
acters
##   for column names in R. [This is a message, not a warning or an error]

res <- results(dds, contrast = c("survival_group", "Survived", "Did Not Survi
ve"))
res_df <- as.data.frame(res) %>% arrange(padj)
head(res_df, 10)

##                      baseMean log2FoldChange     lfcSE      stat       pva
lue
## ENSG00000174343.6    98.00756       3.912808 0.3581378 10.925428 8.712834e
-28
## ENSG00000100604.13  429.18292       4.600498 0.4400102 10.455436 1.383616e
```

```
-25
## ENSG00000109424.4    29.20859       3.560762 0.3642464  9.775696 1.431701e
-22
## ENSG00000135346.9   584.02722       4.255516 0.4502121  9.452247 3.316323e
-21
## ENSG00000248810.2    55.77765       2.719809 0.3039720  8.947564 3.634133e
-19
## ENSG00000188869.13  100.84195       2.390279 0.2781188  8.594453 8.366215e
-18
## ENSG00000134827.8  1148.33487       2.885745 0.3421547  8.434035 3.339469e
-17
## ENSG00000287609.1    60.16780       3.609693 0.4308546  8.377985 5.384158e
-17
## ENSG00000196091.15 1252.22122       2.860985 0.3424480  8.354510 6.570499e
-17
## ENSG00000071991.9    91.88188       3.511698 0.4288931  8.187817 2.660068e
-16
##                             padj
## ENSG00000174343.6  2.919409e-23
## ENSG00000100604.13 2.318041e-21
## ENSG00000109424.4  1.599067e-18
## ENSG00000135346.9  2.778001e-17
## ENSG00000248810.2  2.435378e-15
## ENSG00000188869.13 4.672112e-14
## ENSG00000134827.8  1.598509e-13
## ENSG00000287609.1  2.255087e-13
## ENSG00000196091.15 2.446197e-13
## ENSG00000071991.9  8.913088e-13
```

```
head(res_df, 10)
```

```
##                     baseMean log2FoldChange     lfcSE      stat       pva
lue
## ENSG00000174343.6    98.00756       3.912808 0.3581378 10.925428 8.712834e
-28
## ENSG00000100604.13  429.18292       4.600498 0.4400102 10.455436 1.383616e
-25
## ENSG00000109424.4    29.20859       3.560762 0.3642464  9.775696 1.431701e
-22
## ENSG00000135346.9   584.02722       4.255516 0.4502121  9.452247 3.316323e
-21
## ENSG00000248810.2    55.77765       2.719809 0.3039720  8.947564 3.634133e
-19
## ENSG00000188869.13  100.84195       2.390279 0.2781188  8.594453 8.366215e
-18
## ENSG00000134827.8  1148.33487       2.885745 0.3421547  8.434035 3.339469e
-17
## ENSG00000287609.1    60.16780       3.609693 0.4308546  8.377985 5.384158e
-17
## ENSG00000196091.15 1252.22122       2.860985 0.3424480  8.354510 6.570499e
```

```
-17
## ENSG00000071991.9    91.88188       3.511698 0.4288931  8.187817 2.660068e
-16
##                             padj
## ENSG00000174343.6  2.919409e-23
## ENSG00000100604.13 2.318041e-21
## ENSG00000109424.4  1.599067e-18
## ENSG00000135346.9  2.778001e-17
## ENSG00000248810.2  2.435378e-15
## ENSG00000188869.13 4.672112e-14
## ENSG00000134827.8  1.598509e-13
## ENSG00000287609.1  2.255087e-13
## ENSG00000196091.15 2.446197e-13
## ENSG00000071991.9  8.913088e-13
```

```
res_df[rownames(res_df) == "ENSG00000134827.8", ]
```

```
##                   baseMean log2FoldChange     lfcSE     stat       pvalue
## ENSG00000134827.8 1148.335       2.885745 0.3421547 8.434035 3.339469e-17
##                             padj
## ENSG00000134827.8 1.598509e-13
```

```
normalized_counts <- counts(dds, normalized = TRUE)
```

```
survivor_counts    <- normalized_counts[, dds$survival_group == "Survived"]
nonsurvivor_counts <- normalized_counts[, dds$survival_group == "Did Not Surv
ive"]
```

```
mean_survivors    <- rowMeans(survivor_counts)
mean_nonsurvivors <- rowMeans(nonsurvivor_counts)
```

```
alpha_survivors <- apply(survivor_counts, 1, function(x) {
  m <- mean(x)
  v <- var(x)
  if (m == 0) NA_real_ else (v - m) / m^2
})

alpha_nonsurvivors <- apply(nonsurvivor_counts, 1, function(x) {
  m <- mean(x)
  v <- var(x)
  if (m == 0) NA_real_ else (v - m) / m^2
})
```

```
disp_raw <- mcols(dds)$dispGeneEst
```

```r
disp_shrunken <- dispersions(dds)

nb_var_survivors    <- mean_survivors + alpha_survivors * mean_survivors^2
nb_var_nonsurvivors <- mean_nonsurvivors + alpha_nonsurvivors * mean_nonsurvi
vors^2

log2FC <- log2(mean_survivors / mean_nonsurvivors)

n1 <- ncol(survivor_counts)
n2 <- ncol(nonsurvivor_counts)

se_log2FC <- sqrt(
  (nb_var_survivors / (n1 * mean_survivors^2)) +
  (nb_var_nonsurvivors / (n2 * mean_nonsurvivors^2))
) / log(2)



z_approx <- log2FC / se_log2FC

pval_approx <- 2 * pnorm(-abs(z_approx))

padj_approx <- p.adjust(pval_approx, method = "BH")

# Put into a dataframe
mean_var_df <- data.frame(
  gene                      = rownames(normalized_counts),
  mean_survivors            = mean_survivors,
  mean_nonsurvivors         = mean_nonsurvivors,
  #alpha_survivors = alpha_survivors,
  #alpha_nonsurvivors = alpha_nonsurvivors,
  nb_var_survivors = nb_var_survivors,
  nb_var_nonsurvivors = nb_var_nonsurvivors,
 # DESeqdisp_raw = disp_raw,
  #DESeqdisp_shrunken = disp_shrunken,
  #se_log2FC = se_log2FC,
  z_approx = z_approx,
  pval_approx               = pval_approx#,
#  padj_approx               = padj_approx
)







mean_var_df <- mean_var_df %>%
  arrange(padj_approx)
head(mean_var_df)
```

```
##                                  gene mean_survivors mean_nonsurvivors
## ENSG00000200959.1 ENSG00000200959.1      275.57103         2.0469776
## ENSG00000266019.1 ENSG00000266019.1       52.37592         0.7470221
## ENSG00000174343.6 ENSG00000174343.6      542.15656        13.6597759
## ENSG00000206585.1 ENSG00000206585.1      237.43854         1.9955640
## ENSG00000212402.1 ENSG00000212402.1      146.60177         1.4339506
## ENSG00000202538.1 ENSG00000202538.1     3044.04187        21.0600101
##                   nb_var_survivors nb_var_nonsurvivors z_approx  pval_appr
ox
## ENSG00000200959.1       16689500.5            9.435452 9.139490 6.274728e-
20
## ENSG00000266019.1         444853.2            1.166280 9.106915 8.475765e-
20
## ENSG00000174343.6       20782470.1         1218.797835 9.007188 2.114080e-
19
## ENSG00000206585.1       12265090.8            7.824403 9.008326 2.092256e-
19
## ENSG00000212402.1        4615925.2            3.511889 8.825995 1.084906e-
18
## ENSG00000202538.1     2221912669.2         1721.441088 8.629219 6.177233e-
18
```

```r
mean_var_df[rownames(mean_var_df) == "ENSG00000134827.8",]
```

```
##                                  gene mean_survivors mean_nonsurvivors
## ENSG00000134827.8 ENSG00000134827.8       1881.909          316.8432
##                   nb_var_survivors nb_var_nonsurvivors z_approx  pval_appr
ox
## ENSG00000134827.8        100835233             1250311 4.053256 5.050967e-
05
```

```r
library(ggplot2)
  normalized_counts <- counts(dds, normalized = TRUE)

# Extract that one gene
gene_counts <- normalized_counts["ENSG00000134827.8", ]

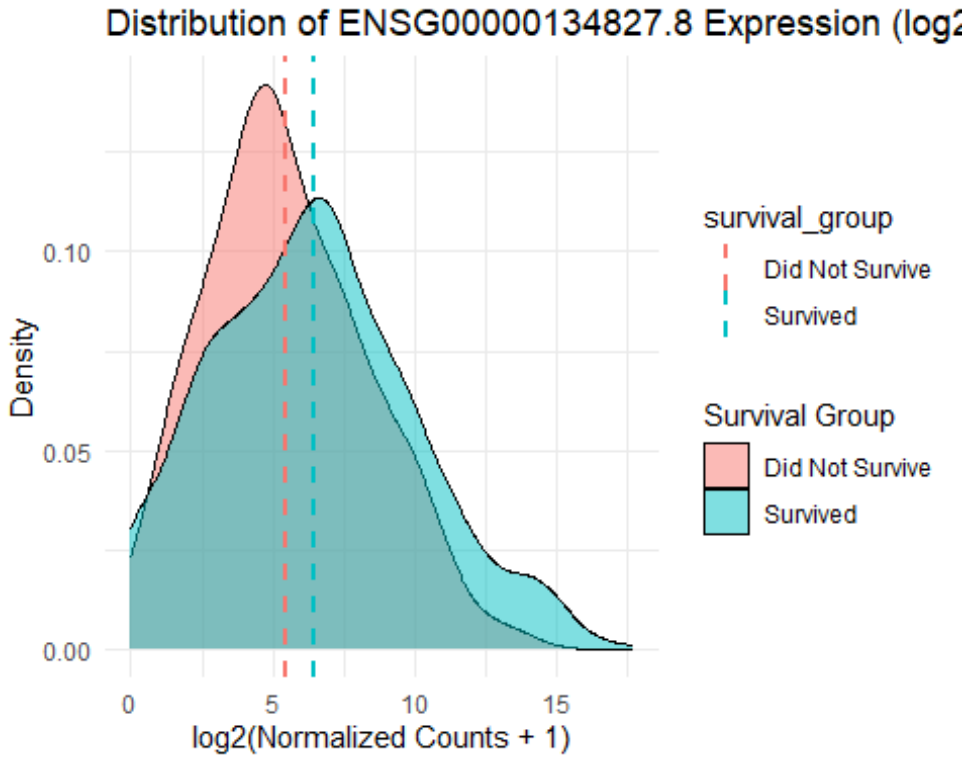
gene_df <- data.frame(
  counts         = gene_counts,
  log_counts     = log2(gene_counts + 1),
  survival_group = dds$survival_group
)



ggplot(gene_df, aes(x = log_counts, fill = survival_group)) +
  geom_density(alpha = 0.5) +
  labs(
    title = "Distribution of ENSG00000134827.8 Expression (log2)",
    x     = "log2(Normalized Counts + 1)",
    y     = "Density",
```

```
    fill  = "Survival Group"
  ) +
  theme_minimal() +
  geom_vline(
    data = gene_df %>% group_by(survival_group) %>%
           summarise(mean = mean(log_counts)),
    aes(xintercept = mean, color = survival_group),
    linetype = "dashed",
    size = 1
  )
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## ℹ Please use `linewidth` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



This will show you the duplicated patients sorted alphabetically so you can clearly see each patient appearing twice with their different sample types side by side like:

```
TCGA-XX-XXXX    Primary Tumor
TCGA-XX-XXXX    Solid Tissue Normal
TCGA-YY-YYYY    Primary Tumor
TCGA-YY-YYYY    Solid Tissue Normal

gene_id <- "ENSG00000134827.8"
```

```r
m1 <- mean_survivors[gene_id]
m2 <- mean_nonsurvivors[gene_id]
nb_v1 <- nb_var_survivors[gene_id]
nb_v2 <- nb_var_nonsurvivors[gene_id]


log2FC <- log2(m1 / m2)


var_log2_s  <- nb_v1  / (m1^2 * log(2)^2)
var_log2_ns <- nb_v2  / (m2^2 * log(2)^2)


pooled_sd <- sqrt((var_log2_s + var_log2_ns) / 2)



power.t.test(
  n           = (2 / (1 / n1 + 1 / n2)),
  delta       = abs(log2FC),
  sd          = pooled_sd,
  sig.level   = 0.05,
  type        = "two.sample",
  alternative = "two.sided"
)

##
##      Two-sample t test power calculation
##
##               n = 142.8738
##           delta = 2.570356
##              sd = 6.526214
##       sig.level = 0.05
##           power = 0.9127031
##     alternative = two.sided
##
## NOTE: n is number in *each* group

power.t.test(
  n           = NULL,
  delta       = abs(log2FC),
  sd          = pooled_sd,
  sig.level   = 0.05,
  power       = 0.99,
  type        = "two.sample",
  alternative = "two.sided"
)
```

```
##
##      Two-sample t test power calculation
##
##               n = 237.8481
##           delta = 2.570356
##              sd = 6.526214
##       sig.level = 0.05
##           power = 0.99
##     alternative = two.sided
##
## NOTE: n is number in *each* group
```